COMP5318/4318 Assignment 2 — Image Classification on PathMNIST

Group 163 · SIDs 530210074, 540868894, 550220550, 550574604 · COMP5318/COMP4318 Machine Learning and Data Mining, Semester 2 2026

1. Introduction

Aim. We compare three classifiers of increasing representational power on PathMNIST, a nine-class set of 28×28 RGB patches of colorectal histology. The three are Gaussian Naive Bayes (GNB), a fully connected multilayer perceptron (MLP) and a convolutional neural network (CNN). Two untuned reference baselines anchor the scale: a majority-class Dummy and a Nearest Centroid classifier. For each assessed model we design the architecture or setup and tune at least three hyperparameters (one for GNB, which has a single continuous hyperparameter) on a held-out validation set. We then retrain with the selected configuration and compare all five models on the untouched test set. The comparison covers accuracy, Macro-F1 and balanced accuracy, together with runtime, parameter count, per-class behaviour and the robustness of results across hardware.

Importance. Tissue-type classification is a building block of computational pathology. Automatically separating tumour epithelium, stroma, mucus, debris and normal mucosa on whole-slide images supports tumour-content estimation and prognostic scoring of colorectal cancer [3], tasks that are slow and variable when done manually. Beyond this dataset, the study shows *why* model families differ on image data. GNB tests what can be done with per-pixel statistics alone. The MLP adds learned non-linear interactions but no notion of space. The CNN adds locality and weight sharing. Measuring the gain at each step, its cost in runtime and parameters, and how stable it is, helps choose a model for a practical setting where interpretability, compute budget and reproducibility matter as well as accuracy. While re-running our pipeline on different machines we also found that neural-network results shift measurably with the CPU, even with pinned software. We document this because it affects how confidently small performance differences can be reported.

2. Data

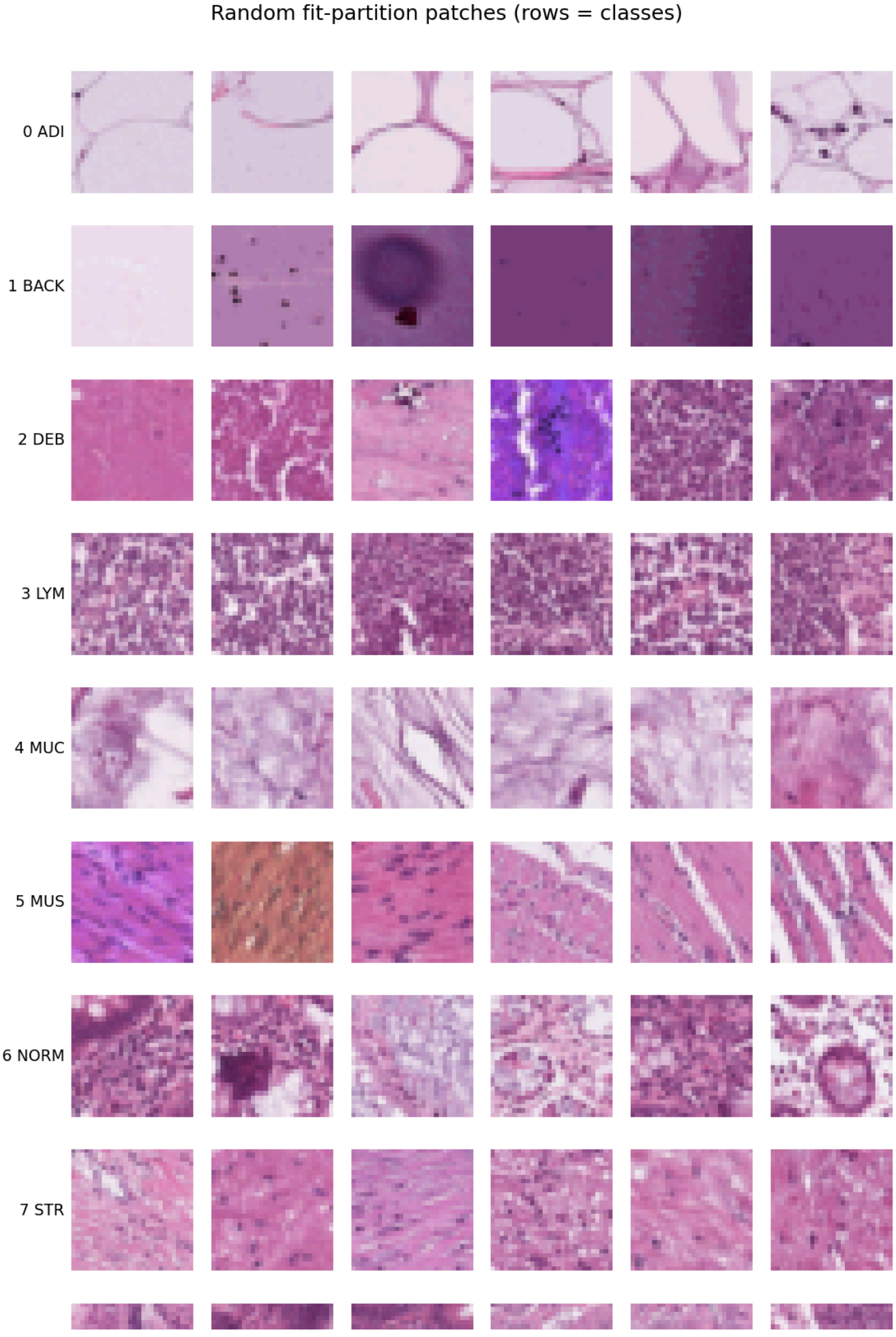
2.1 Data description and exploration

The course provides 40,000 PathMNIST images (32,000 training, 8,000 test), each a 28×28×3 `uint8` RGB array, with nine tissue classes. PathMNIST is part of MedMNIST v2 [1], [2]. Its images derive from the NCT-CRC-HE-100K dataset of Kather et al. [3]:

224×224-pixel patches (0.5 µm/pixel) cut from 86 H&E-stained colorectal cancer slides, which MedMNIST resized to 28×28. Each image is a crop of continuous tissue, not a photograph of an object. All exploration below uses only the 25,600-image fit partition (see "Development split" below); the test set was inspected only for its label counts.

Label	Class	Abbrev.	Train n	Train %	Mean RGB (0–1)	Brightness s.d. between images
0	Adipose	ADI	3,490	10.9	0.85 / 0.76 / 0.85	0.036
1	Background	BACK	3,431	10.7	0.61 / 0.43 / 0.61	**0.194**
2	Debris	DEB	3,505	11.0	0.75 / 0.49 / 0.70	0.060
3	Lymphocytes	LYM	3,656	11.4	0.69 / 0.49 / 0.66	0.044
4	Mucus	MUC	2,950	9.2	0.77 / 0.60 / 0.75	0.052
5	Smooth muscle	MUS	4,290	13.4	0.78 / 0.52 / 0.72	0.058
6	Normal colon mucosa	NORM	2,728	8.5	0.73 / 0.52 / 0.69	0.061
7	Cancer-associated stroma	STR	3,253	10.2	0.77 / 0.53 / 0.72	0.046
8	Colorectal adenocarcinoma epithelium	TUM	4,697	14.7	0.71 / 0.48 / 0.66	0.057

Train n and % are over all 32,000 training images. The test set has identical proportions (within 0.01 percentage points). Colour statistics are computed on the fit partition.


Random fit-partition patches (rows = classes)
0 ADI
1 BACK
2 DEB
3 LYM
4 MUC
5 MUS
6 NORM
7 STR

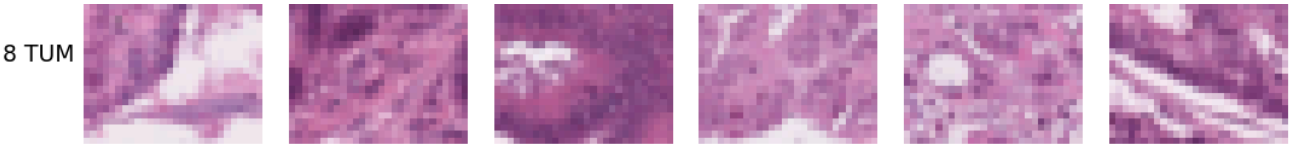


Figure 1. Six random fit-partition patches per class. Within-class variation is large (e.g. BACK), while DEB, MUS, NORM, STR and TUM share similar pink-purple tones.

First, we explored the data interactively with a small Streamlit dashboard that we built with ChatGPT's help. It shows class-balanced galleries, RGB histograms, per-class colour statistics and PCA scatter plots. We then reproduced each observation in the notebook with fixed seeds. Five findings shaped the later design:

1. **Moderate imbalance.** TUM is 1.72 times as frequent as NORM, and a majority-class predictor already reaches 0.147 accuracy. We therefore select models by **Macro-F1**, which weights every class equally.
2. **Colour helps, but only at the extremes.** ADI is bright and smooth (fat vacuoles). LYM and TUM are dark and purple (dense nuclei). BACK has three times the between-image brightness variation of any other class, because "background" can be bright glass or dark empty regions. The remaining classes (DEB, MUC, MUS, NORM, STR) share pink-purple means within about 0.05 of each other. Colour alone should therefore not separate them.
3. **No centred object.** The mean intensity varies by less than 0.005 across the 28×28 positions. The same texture can appear anywhere in the patch, which favours translation-equivariant convolution over the MLP's position-specific weights.
4. **Strong, local pixel dependence.** Adjacent pixels correlate at $r \approx 0.87$. The three channels of one pixel correlate at $r \approx 0.89$–0.95. Correlation falls to about 0.64 at 5 pixels and 0.57 at 14 pixels. The residual reflects a shared brightness component. GNB's conditional-independence assumption is therefore strongly violated, and the excess dependence is local, which is what 3×3 convolutions model.
5. **Redundancy and overlapping classes.** The first principal component alone (overall brightness) explains 55 % of the pixel variance. About 49 components explain 80 % and about 157 explain 90 %. In two dimensions only the extreme classes separate. The closest class-mean images are MUS–STR and LYM–TUM, followed by pairs among DEB, NORM and STR. We therefore expected errors to cluster in these pairs for every model.

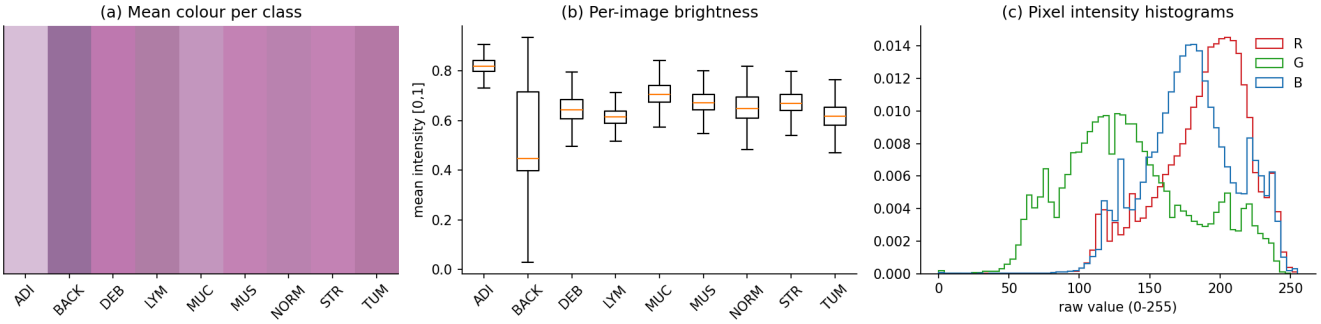


Figure 2. (a) Mean colour per class, (b) per-image brightness, (c) pixel-intensity histograms by channel (fit partition).

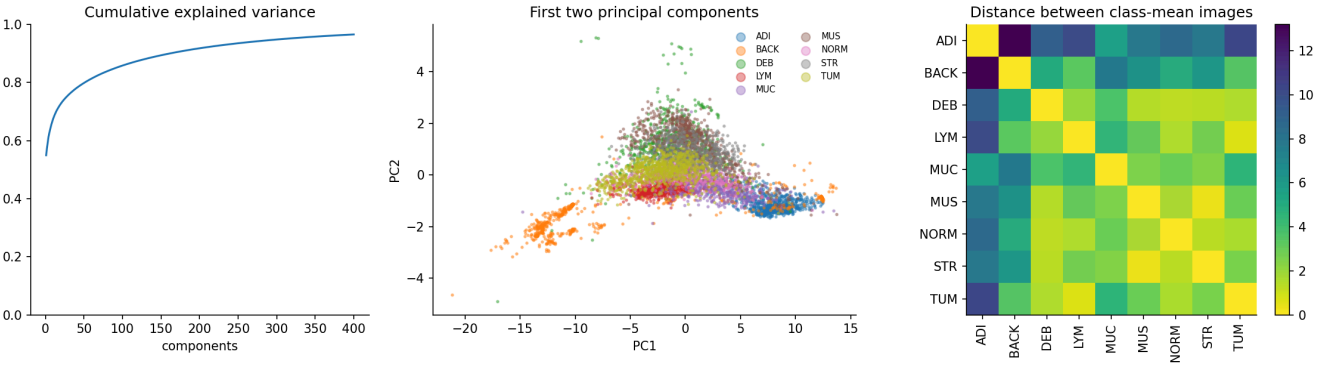


Figure 3. Left: cumulative explained variance. Centre: first two principal components (6,000 fit images). Right: Euclidean distances between class-mean images; the darkest off-diagonal cells (MUS–STR, LYM–TUM) predict the main confusions.

Development split. We held out a stratified 20 % of the training set as a validation set (25,600 fit / 6,400 validation, seed 5318), shared by all models. A single hold-out was preferred to 5-fold cross-validation because each neural trial costs 15–95 s on our 2-core CPU. With 6,400 images, the standard error of a validation accuracy near 0.7 is about 0.006, small enough to rank our configurations. The split's index arrays are fingerprinted with SHA-256, so any accidental change of split is detected.

2.2 Pre-processing

- **Scaling to [0, 1].** We cast every image to `float32` and divide by the constant 255. Neural networks train poorly on raw 0–255 inputs: initial pre-activations are large and gradients badly scaled, so Adam's first steps saturate ReLUs. Because the divisor is fixed rather than estimated, no information from validation or test data enters preprocessing. GNB and Nearest Centroid are unaffected by a common positive scale, so all models can share the same inputs.
- **Flattening (Dummy, Nearest Centroid, GNB, MLP).** We reshape to 2,352-dimensional vectors, because these models take a feature vector. No value is lost, but the 2-D neighbourhood is discarded. That is exactly the property the comparison with the CNN examines.

- **Image tensors (CNN).** The CNN keeps the (28, 28, 3) layout, since convolution requires it.

We considered and deliberately omitted the following:

- *Per-feature standardisation.* GNB is invariant to it, since it fits its own per-feature mean and variance. The pixels already share one bounded scale.
- *PCA.* It would make the CNN inapplicable and the models' inputs non-comparable. We revisit PCA as a decorrelating step for GNB in future work.
- *Grayscale conversion.* Finding 2 shows that colour carries class information.
- *Resizing.* 28×28 is already at the limit where texture survives.
- *Data augmentation (rotations and flips).* It is plausible for rotation-invariant tissue, but it would confound the model comparison and multiply training time.
- *Stain normalisation* [4]. No slide or centre metadata are provided, and part of the colour variation is genuine tissue difference.
- *Class re-weighting.* The imbalance is mild, and Macro-F1 selection already protects the small classes.

3. Methods

3.1 Theory

Reference baselines. *Dummy* always predicts the most frequent training class. It is the floor that any model using the image must beat. *Nearest Centroid* stores one mean image per class and assigns a test image to the class with the closest mean in Euclidean distance. It is a one-prototype version of the nearest-neighbour idea, with a linear decision boundary between each pair of classes.

Gaussian Naive Bayes (algorithm from weeks 1–6). GNB is a generative classifier. It applies Bayes' rule with class priors P(k), modelling each feature as a class-conditional Gaussian and assuming the features are independent given the class:

```
\hat{y} = \arg\max_{k} \Big[ \log P(k) + \sum_{j=1}^{2352} \log
\mathcal{N}\big(x_j \mid \mu_{kj},\, \sigma^2_{kj} + \varepsilon\big) \Big]
```

Training is closed-form: a per-class mean and variance for each of the 2,352 features. The smoothing term ε = `var_smoothing` × (largest feature variance) keeps near-constant pixels from producing infinite log-likelihoods [5]. We chose GNB for four reasons. (i) As a generative, non-iterative model it contrasts as much as possible with the two discriminative, gradient-trained networks. (ii) Its cost is linear in samples and features, so fitting 32,000×2,352 values takes about a second, whereas k-NN must compute 256

million distances at prediction time. (iii) It is interpretable: each class is a mean and a variance image. (iv) Finding 4 lets us state in advance which of its assumptions will fail, so its result tests how much pixel dependence matters.

Multilayer perceptron. An MLP stacks affine layers and element-wise non-linearities, $h = \mathrm{ReLU}(W_1 x + b_1)$, followed by $p = \mathrm{softmax}(W_2 h + b_2)$. It is trained end to end by back-propagating the cross-entropy loss, here with the Adam optimiser [6]. With enough hidden units it can approximate any continuous decision function, and it makes no independence assumption: correlated pixels are combined through learned weights. However, every weight is tied to one absolute input position. A pattern shifted by one pixel meets completely different weights, so the network must see each texture at each location to learn it.

Convolutional neural network. A CNN replaces the dense input layer with small filters that are convolved over the image [7]. Two structural priors follow. **Local connectivity**: each unit sees only a 3×3 neighbourhood, matching the local dependence in finding 4. **Weight sharing**: the same filter is applied at every position, making the feature maps translation-equivariant (finding 3). Max-pooling then provides limited translation invariance and reduces resolution. Stacking layers grows the receptive field, so later filters respond to larger motifs built from earlier edges and blobs. Because parameters are shared, a convolutional layer needs far fewer weights than a dense layer over the same input.

3.2 Strengths and weaknesses

Property	Nearest Centroid	GaussianNB	MLP	CNN
Decision boundary	Linear (equal spherical classes)	Quadratic (per-class diagonal covariance)	Arbitrary non-linear	Arbitrary non-linear over local features
Key assumption	Classes are compact around their means	Features independent given the class	None about structure, but no spatial prior	Local, translation-equivariant patterns
Fit for this data	Fails on the overlapping pink classes (finding 2)	Violated: $r \approx 0.9$ between neighbours	Must learn every position separately	Matches findings 3 and 4

Property	Nearest Centroid	GaussianNB	MLP	CNN
Training cost	One pass; well under 1 s	One pass; about 1 s	Iterative; about 10 epochs	Iterative; about 10 epochs, more FLOPs per image
Parameters	21,168 (9 means)	42,345	302k–605k	52k–106k
Overfitting risk	Very low (high bias)	Low variance, high bias; over-confident posteriors	High: many weights per sample	Moderate: weight sharing regularises
Interpretability	High: mean images	High: mean and variance images	Low: distributed weights	Low to medium: first-layer filters can be inspected
Sensitivity to hardware numerics	Negligible (closed form)	Negligible (closed form)	Present (SGD path)	Present (SGD path)

We therefore expected CNN > MLP > GNB on Macro-F1. We expected GNB to be fastest but over-confident, because adding 2,352 correlated log-likelihoods counts the same evidence many times [8]. We expected the MLP to need many parameters for a modest gain. And because the CNN's parameters sit mostly in its dense head, we expected it to be smaller than the MLP but slower per epoch, since convolution adds computation per image.

3.3 Architecture and hyperparameters

Shared training setup. Both networks are trained with Adam and sparse categorical cross-entropy on batches of 128, from identical seeded initial weights and a seeded data order. During tuning we train for at most 12 epochs, with early stopping (patience 3) on validation Macro-F1, and restore the best epoch's weights. We monitor Macro-F1 rather than validation loss because it is the selection criterion, and cross-entropy can keep falling simply because predictions on already-correct classes become more confident. Macro-F1 is computed exactly per epoch from an accumulated confusion matrix, not by averaging per-batch F1, which would be biased.

MLP architecture. Input(2352) → Dense(h, ReLU) → Dropout(p) → Dense(9, softmax). We used one hidden layer because the first layer already holds (2352+1)·h weights. Extra depth would add parameters and runtime without addressing the MLP's real limitation, the lack of a spatial prior.

CNN architecture. Conv(f, 3×3, same, ReLU) → MaxPool 2×2 → Conv(2f, 3×3, same, ReLU) → MaxPool 2×2 → Flatten → Dense(64, ReLU) → Dropout(p) → Dense(9, softmax).

- **3×3 kernels** are the smallest that capture neighbour dependence. Two of them, with 2×2 pooling in between, give an 8×8 receptive field (10×10 after the second pooling), enough for nuclei and glandular fragments in a 28-pixel patch.
- **"Same" padding and stride 1** keep border texture. A stride of 2 or larger kernels would discard the fine structure that separates DEB, STR and MUS.
- **Pooling** halves the resolution (28→14→7), and the channels double (f→2f) to compensate.
- **Dense head.** The Dense(64) layer holds about 95 % of the 106,089 parameters at f = 16 (100,416 of them). This is why the CNN is smaller than the MLP despite being more expressive.

Search method. We ran a full-factorial grid on the fixed validation split, choosing by validation Macro-F1 and breaking ties by fewer parameters, then by earlier trial. Two values per hyperparameter, each placed on either side of a plausible regime boundary, give a balanced 2^3 design. From it we can read main effects and interactions, which a random or Bayesian search of the same size would not provide. Eight trials per network also fit our CPU budget at full data size. Every trial records fit time, mean time per epoch, best epoch and parameter count. The test set is never used.

Model	Hyperparameter	Values	What it controls and our expectation
GNB	`var_smoothing`	1e-12 … 1 (8 values, log-spaced)	Variance floor relative to the largest variance (≈ 0.034). We expected a flat plateau while ε · 0.034 is much smaller than the smallest variance (≈ 0.015), and a decline once it is comparable (ε ⩾ 0.1).
MLP	Hidden units h	128, 256	Capacity and parameters (302k vs 605k). We expected 256 to help, with about twice the time per epoch.

Model	Hyperparameter	Values	What it controls and our expectation
MLP	Learning rate	1e-3, 3e-4	Step size. 1e-3 converges faster within 12 epochs, while 3e-4 is steadier but may not converge in time.
MLP	Dropout p	0, 0.3	Regularisation. We expected little benefit for a small MLP trained for 12 epochs, with a possible risk of underfitting.
CNN	Base filters f (second layer 2f)	8, 16	Number of local pattern detectors (52k vs 106k parameters). Convolutional cost grows roughly with f^2. We expected 16 to help across nine textures.
CNN	Learning rate	1e-3, 3e-4	As for the MLP. We expected 1e-3 to work well on the CNN's smoother loss surface.
CNN	Dropout p (dense head)	0, 0.3	Regularises the 64-unit head where the parameters sit. We expected it to help more than in the MLP, because the CNN fits faster and can start to overfit.

No GNB setting other than ε changes the fitted likelihood, so ε is the hyperparameter that matters. We kept the empirical priors because their log-ratios (at most 0.54) are negligible beside a likelihood summed over 2,352 terms. Kernel size, depth and batch size were fixed for both networks to keep the search to eight trials of 15–95 s each.

Final training protocol. Each selected configuration is retrained from scratch on all 32,000 training images for a fixed number of epochs and evaluated once on the 8,000 test images. Without a validation set, early stopping is impossible, so the epoch count is carried over from the search (Section 4.1).

4. Results and discussion

All numbers below come from one fresh-kernel execution of the submitted notebook, with the pinned software stack, on a 2-core Intel Xeon (AVX-512) CPU. A second complete execution on the same machine reproduced every metric exactly. Section 4.4 discusses

how the neural-network results change on other CPUs.

4.1 Hyperparameter tuning

GaussianNB. Validation Macro-F1 is identical (0.3749) for every ε from 1e-12 to 1e-6. It slips slightly at 1e-4 and 1e-2 (0.3748 and 0.3719), falls at 0.1 (0.3597), and collapses at 1 (0.2664). This is exactly the predicted shape: smoothing is negligible until ε × 0.034 approaches the smallest pixel variance (0.015). Beyond that point it erases the class-specific variance differences that distinguish GNB from a common-variance model. Fit time (0.55–0.68 s) is unrelated to ε, because ε is added after the variances are computed. We chose ε = 1e-6, the largest value on the plateau, as a principled tie-break. The flat plateau also shows that GNB's weakness is its assumption, not its regularisation. Its best validation score is below the parameter-free Nearest Centroid (0.405).

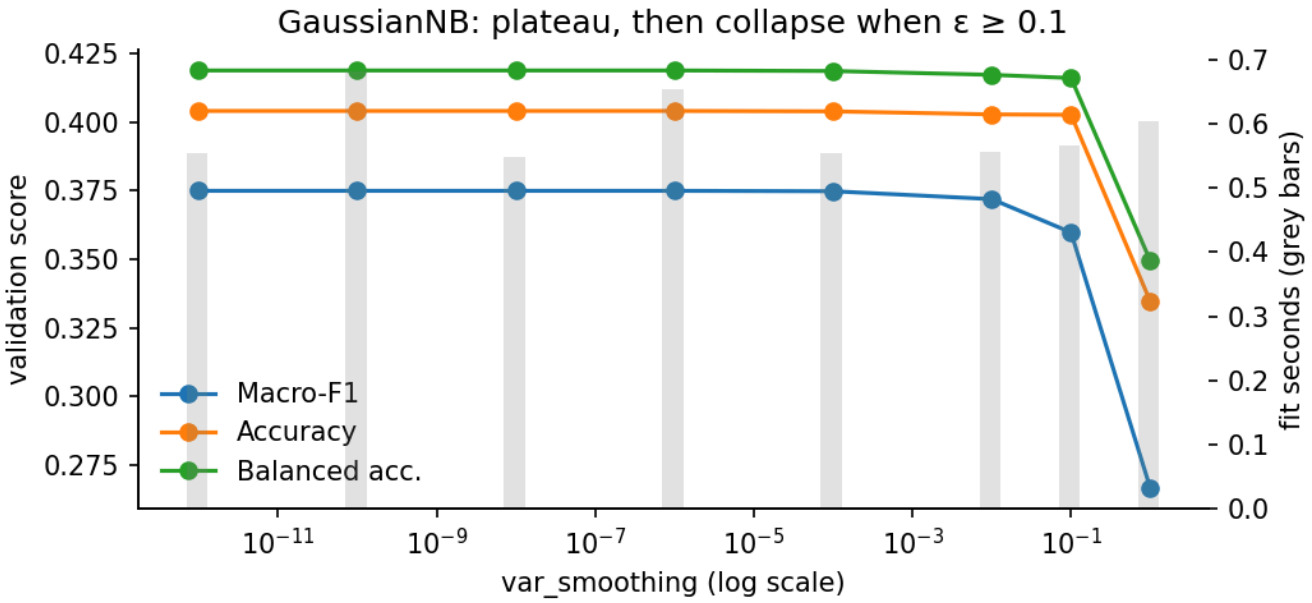


Figure 4. GaussianNB search: a flat plateau up to ε = 1e-6, then decline.

MLP (8 trials, total search time 216.8 s):

Trial	Hidden units	Learning rate	Dropout	Val Macro-F1	Val accuracy	Best / run epochs	Fit time (s)	Time per epoch (s)
1	128	1e-3	0.0	0.385	0.428	5 / 8	19.1	1.99
2	128	1e-3	0.3	0.034	0.150	3 / 6	15.5	2.26
3	128	3e-4	0.0	0.404	0.439	9 / 12	23.4	1.71
4	128	3e-4	0.3	0.337	0.412	11 / 12	24.7	1.81
5	**256**	**1e-3**	**0.0**	**0.428**	**0.446**	**9 / 12**	**36.1**	**2.68**
6	256	1e-3	0.3	0.341	0.380	4 / 7	21.7	2.75
7	256	3e-4	0.0	0.406	0.429	10 / 12	38.3	2.88

Trial	Hidden units	Learning rate	Dropout	Val Macro-F1	Val accuracy	Best / run epochs	Fit time (s)	Time per epoch (s)
8	256	3e-4	0.3	0.358	0.417	7 / 10	27.6	2.47

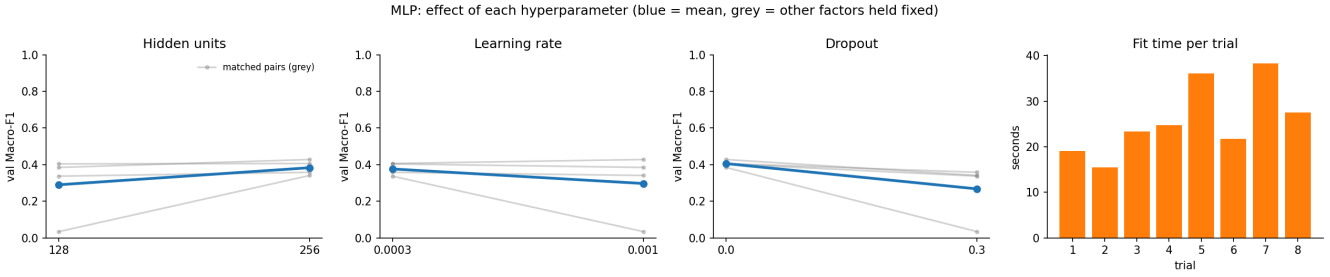


Figure 5. MLP search. Blue: mean validation Macro-F1 per value. Grey: matched pairs with the other two factors fixed. Right: fit time per trial.

- **Width** helped in all four matched pairs (mean 0.383 vs 0.290), as predicted. Time per epoch rose by only about 1.4× on average (1.2–1.7× across matched pairs) rather than 2×, because part of each epoch is fixed pipeline and validation overhead.
- **Dropout** hurt in all four pairs (0.267 vs 0.406), more than our "little benefit" prediction. The MLP is *underfitting*: training and validation loss stay close (1.508 vs 1.509 at the best epoch), and adding noise to an underfitting model only slows learning [9].
- **Learning rate** shows an interaction. On average 3e-4 looks better (0.376 vs 0.297), but the best model uses 1e-3. The average is distorted by **trial 2**, which collapsed: its loss stayed at ≈ 2.18 ≈ ln 9, i.e. it predicted almost one class for every image. A plausible cause is that all-positive, un-centred inputs combined with large early Adam steps and dropout noise silenced most of the 128 ReLUs ("dying ReLU"). We kept the trial because it reproduced on every machine. It is a genuine instability of this configuration, not a bug.
- Even the best MLP (0.428) is only marginally above Nearest Centroid (0.405), despite 605k parameters.

CNN (8 trials, total search time 534.7 s):

Trial	Base filters	Learning rate	Dropout	Val Macro-F1	Val accuracy	Best / run epochs	Fit time (s)	Time per epoch (s)
1	8	1e-3	0.0	0.707	0.722	12 / 12	50.4	3.77
2	8	1e-3	0.3	0.706	0.718	12 / 12	49.2	3.54
3	8	3e-4	0.0	0.672	0.687	12 / 12	55.2	4.01
4	8	3e-4	0.3	0.660	0.680	12 / 12	48.0	3.53
5	16	1e-3	0.0	0.720	0.728	7 / 10	55.9	4.94
6	**16**	**1e-3**	**0.3**	**0.728**	**0.741**	**11 / 12**	**77.9**	**5.87**
7	16	3e-4	0.0	0.700	0.718	12 / 12	95.8	6.93

Trial	Base filters	Learning rate	Dropout	Val Macro-F1	Val accuracy	Best / run epochs	Fit time (s)	Time per epoch (s)
8	16	3e-4	0.3	0.695	0.710	12 / 12	86.1	6.56

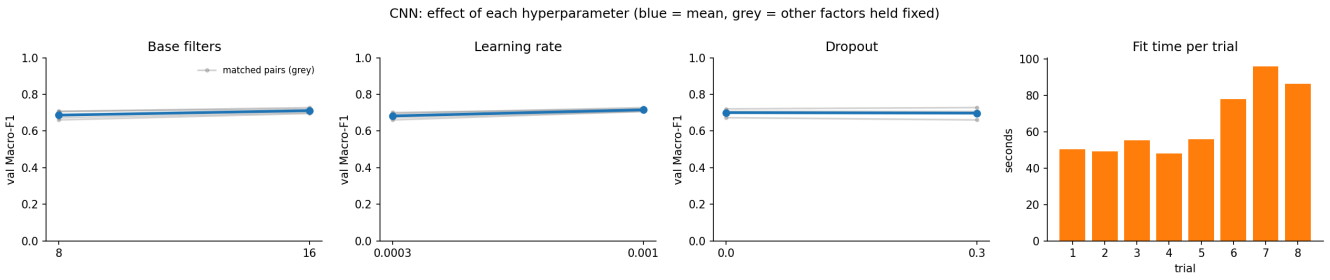


Figure 6. CNN search, same layout as Figure 5.

- **16 filters** beat 8 in every matched pair (0.711 vs 0.687), at about 1.3–1.9× the time per epoch. Convolution cost grows with the filter count, even though most of the extra parameters sit in the dense head.
- **Learning rate 1e-3** beat 3e-4 in every pair (0.715 vs 0.682). However, six of the eight trials, including every 3e-4 trial, peaked at the 12-epoch cap. Those models were still improving, so this comparison partly measures convergence speed within our budget rather than final quality.
- **Dropout** helped only in the strongest setting (16 filters at 1e-3: 0.728 vs 0.720). There the dropout-free trial peaked at epoch 7 and early-stopped. This matches our prediction that the CNN, unlike the MLP, learns fast enough to start overfitting, so regularising its 100k-parameter dense head pays off. The gain is small, and we treat it as a weak preference.
- The *worst* CNN trial (0.660) beats the *best* MLP trial (0.428) by 0.23 with about 6–12× fewer parameters. This is the clearest evidence that inductive bias, rather than capacity, is what matters here.

Epoch selection. Every execution of the search, on every machine, selected the same MLP and CNN configurations. The arg-max epoch, however, varied: 10 / 11 on the development laptops, 9 / 12 in Docker on an AMD Ryzen, and 9 / 11 in this run. The epoch curves (Figure 7) show why. With a constant learning rate, validation Macro-F1 oscillates by several points between consecutive epochs (MLP epochs 7–10: 0.354, 0.363, 0.428, 0.403; CNN epochs 9–12: 0.702, 0.692, 0.728, 0.727). The arg-max of such a curve is fragile. We kept the epoch counts frozen before testing (MLP 10, CNN 11), because re-selecting after seeing test results would leak test information.

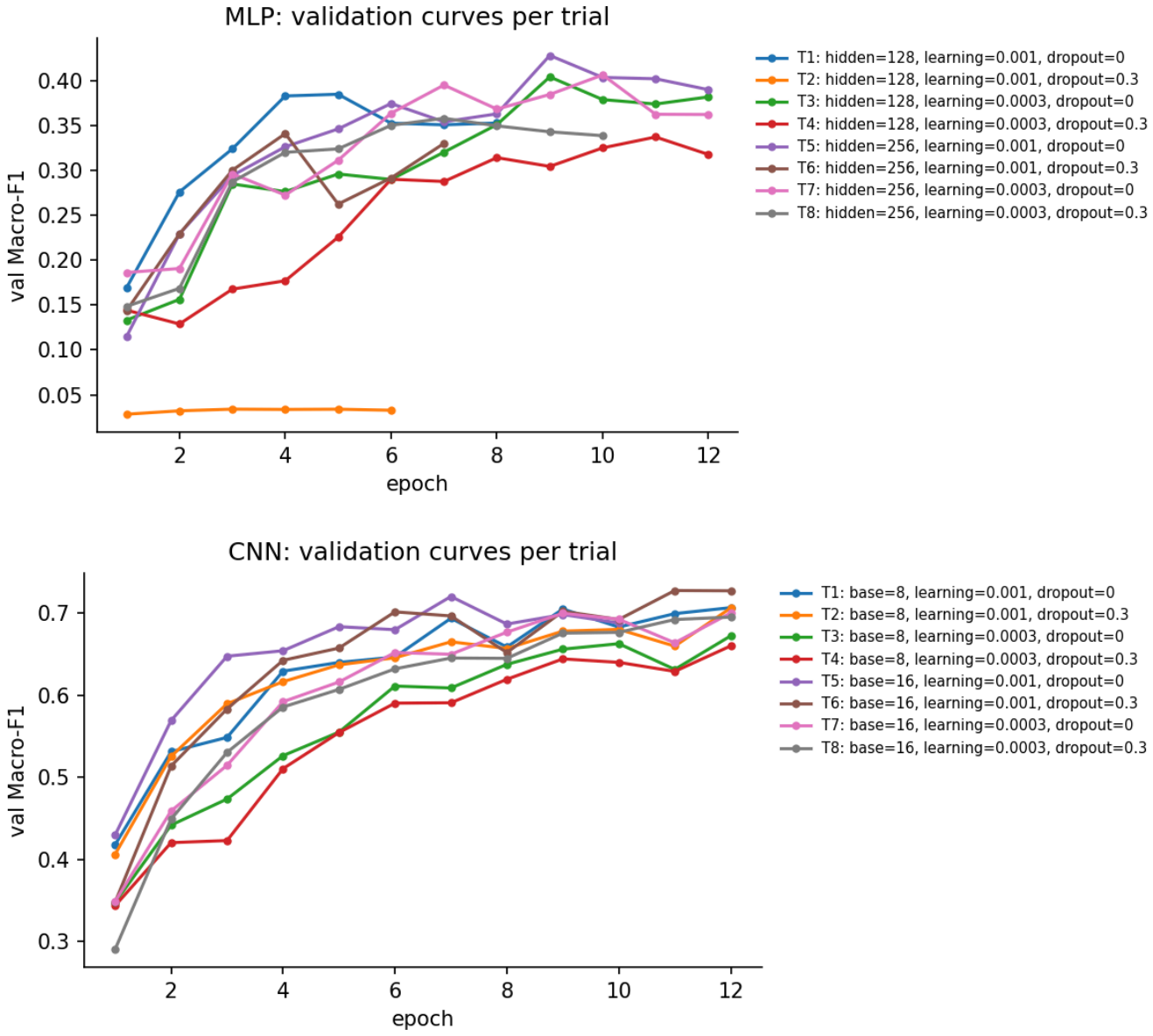


Figure 7. Validation Macro-F1 per epoch for every MLP (top) and CNN (bottom) trial.

4.2 Final model comparison

Each model was retrained on all 32,000 training images with its selected configuration and evaluated once on the 8,000 test images.

Model	Selected hyperparameters	Val Macro-F1	Test accuracy	Test Macro-F1	Test balanced acc.	Fit time (s)	Predict time (s)	Parameters
Dummy	most frequent	0.028	0.147	0.028	0.111	0.002	<0.001	0
Nearest Centroid	Euclidean	0.405	0.430	0.410	0.433	0.14	0.11	21,168
GaussianNB	var_smoothing = 1e-6	0.375	0.415	0.386	0.431	0.67	1.64	42,345

Model	Selected hyperparameters	Val Macro-F1	Test accuracy	Test Macro-F1	Test balanced acc.	Fit time (s)	Predict time (s)	Parameters
MLP	h = 256, lr = 1e-3, dropout = 0, 10 epochs	0.428	0.485	0.443	0.473	33.8 (3.4 / epoch)	0.56	604,681
CNN	**f = 16, lr = 1e-3, dropout = 0.3, 11 epochs**	**0.728**	**0.708**	**0.683**	**0.688**	**86.6 (7.9 / epoch)**	**0.98**	**106,089**

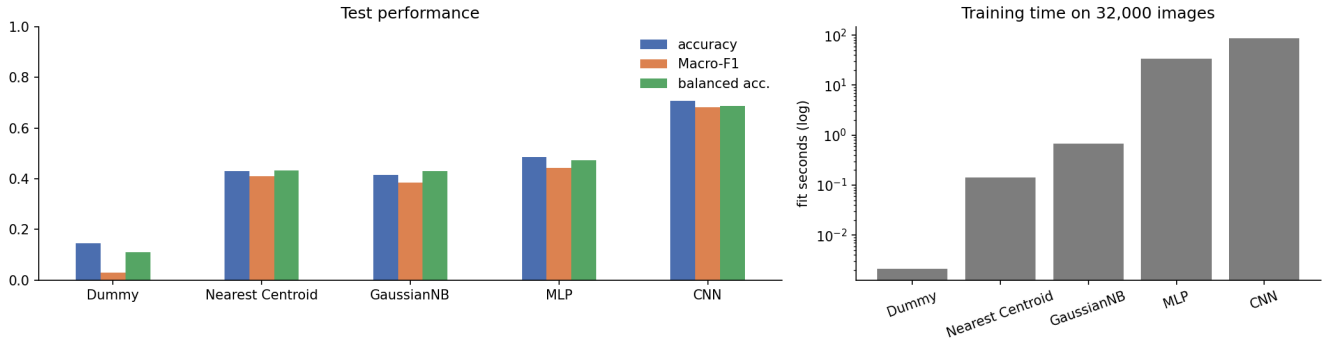


Figure 8. Test performance (left) and training time on 32,000 images, log scale (right).

The ranking CNN ≫ MLP > Nearest Centroid > GNB ≫ Dummy matches the theory in Section 3.2, with one exception. **GNB loses to Nearest Centroid** (0.386 vs 0.410 Macro-F1). Its extra flexibility, per-class per-pixel variances, is harmful when 2,352 strongly correlated log-likelihoods are summed as if independent: small variance mismatches are counted hundreds of times. The symptom is severe over-confidence. **96.9 % of GNB's test predictions have a posterior above 0.999**, yet only 41.5 % are correct.

The **MLP's 605k parameters buy only +0.03 Macro-F1** over Nearest Centroid. Its accuracy (0.485) hides a complete failure on normal mucosa: NORM F1 is 0.02 (recall 0.01). The **CNN** raises Macro-F1 by a further 0.24 with 5.7× fewer parameters than the MLP.

Runtime. Runtime follows computation, not parameters. The CNN is ≈ 2.3× slower per epoch than the MLP. Its two convolutions reuse every weight at 784 or 196 positions, giving about 1.2 M multiply-adds per image (plus 0.1 M in the dense head), against about 0.6 M for the MLP's first layer. GNB trains in under a second, a single pass of means and variances. Its prediction (1.6 s) is the slowest, because it evaluates 9 × 2,352 Gaussian log-densities per image. All models predict 8,000 images in under 2 s, so inference cost is not a deciding factor here.

Validation vs test. The MLP scored 0.428 on validation and 0.443 on test; the CNN 0.728 and 0.683. These gaps are not clean generalisation estimates. The final models had 25 % more data, a fixed number of epochs, and a different evaluation set, and the CNN gap is

within the cross-platform spread (Section 4.4). The final CNN's training accuracy (0.65 with dropout active) is below its test accuracy (0.71), which indicates underfitting rather than overfitting.

4.3 Per-class behaviour and types of error

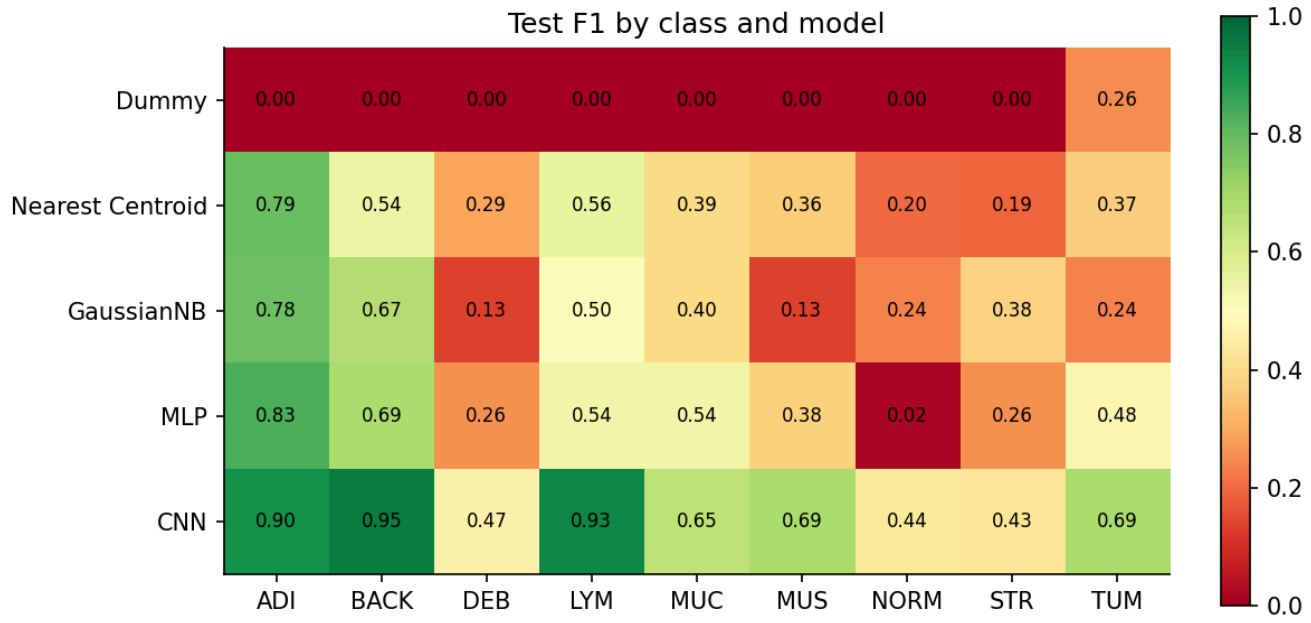


Figure 9. Test F1 by class (columns) and model (rows).

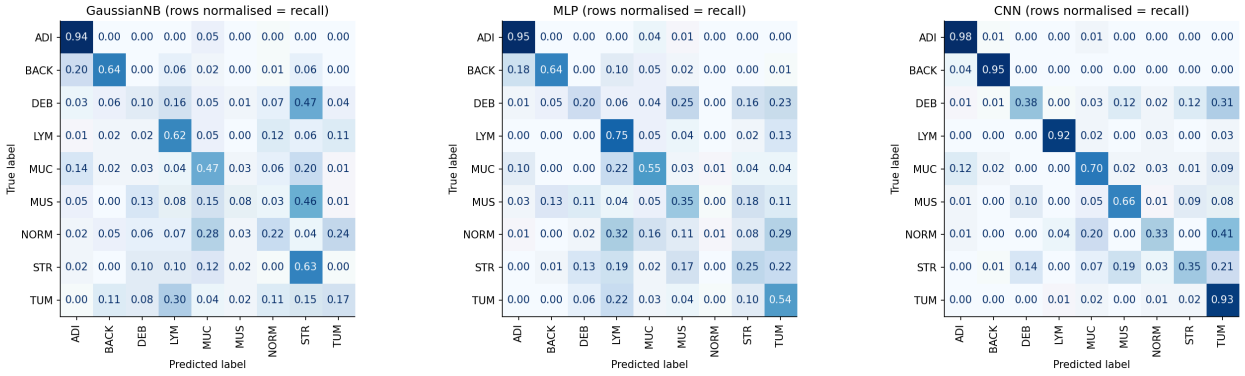


Figure 10. Row-normalised test confusion matrices (diagonal = recall) for GNB, MLP and CNN.

Model	Largest confusions (true → predicted, count)	Precision vs recall pattern
GaussianNB	MUS→STR 497, DEB→STR 409, TUM→LYM 355, NORM→MUC 193	STR is an "attractor": recall 0.63, precision 0.27. MUS recall falls to 0.08.

Model	Largest confusions (true → predicted, count)	Precision vs recall pattern
MLP	TUM→LYM 255, NORM→LYM 219, DEB→MUS 217, NORM→TUM 199	LYM recall 0.75 at precision 0.42. NORM is almost never predicted (recall 0.01).
CNN	NORM→TUM 281, DEB→TUM 273, STR→TUM 169, STR→MUS 154	TUM recall 0.93 at precision 0.55. NORM (0.33) and STR (0.35) recall are lowest.

The confusions follow the class-centroid structure found in the exploration. For GNB, MUS–STR is the closest centroid pair and is its single largest confusion, and LYM–TUM is the second closest. The CNN improves every class, most of all the **texture-defined** ones (MLP → CNN F1): BACK 0.69 → 0.95, LYM 0.54 → 0.93, MUS 0.38 → 0.69. These are local patterns that one shared 3×3 filter can find anywhere in the patch.

Its remaining errors flow into **TUM**. Normal mucosa and adenocarcinoma are both glandular epithelium, distinguished mainly by gland architecture over hundreds of micrometres, and much of that is lost at 28×28 pixels with a receptive field of about 10×10 pixels. Debris and stroma are heterogeneous mixtures that contain tumour-like fragments. The precision-recall imbalance matters in practice: a CNN that over-calls tumour would overestimate tumour content.

The CNN's confidence is informative. Its mean top-class probability is 0.75 when correct and 0.50 when wrong, so low-confidence patches could be routed for review. GNB's posteriors carry no such signal.

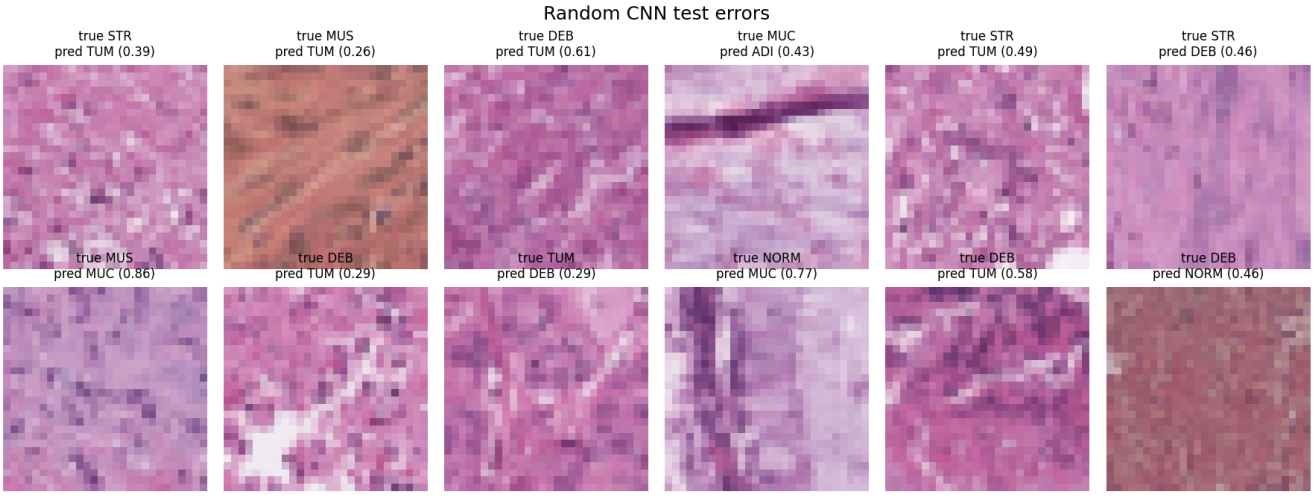


Figure 11. Random CNN test errors (true / predicted class, predicted probability).

4.4 Reproducibility across hardware

We ran the identical pipeline (same data hashes, split, seed, code and hyperparameters) on several machines during development. The software was pinned and run in Docker on all but the two development laptops.

Platform	CPU	MLP test acc.	MLP test Macro-F1	CNN test acc.	CNN test Macro-F1
Windows 11 laptop (development)	Intel x86-64	0.496	0.449	–	–
macOS laptop (development)	Apple Silicon (arm64)	–	–	0.720	0.698
Docker, workstation	AMD Ryzen 9 5900HS (AVX2)	0.489	0.439	0.739	0.723
Docker, GitHub-hosted runner, 1 of 3 runs	not logged	0.489	0.439	0.739	0.723
Docker, GitHub-hosted runner, 2 of 3 runs	not logged	0.485	0.443	0.708	0.683
Submitted notebook run	**Intel Xeon (AVX-512)**	**0.485**	**0.443**	**0.708**	**0.683**

Dummy, Nearest Centroid and GNB were bit-identical everywhere (Macro-F1 0.028438 / 0.410469 / 0.385748). The networks were not. CNN test Macro-F1 spans 0.683–0.723 (0.040), and MLP Macro-F1 spans 0.439–0.449 (0.010). The final MLP's training loss matched the Windows run to four decimals for two epochs (2.0129, 1.7905) before drifting apart.

The outcomes are **not random scatter: they cluster by CPU.** Our Intel AVX-512 run reproduced two earlier CI runs to every printed digit. The third CI run reproduced the AMD workstation exactly. Hosted runners are drawn from mixed Intel and AMD pools, and the CNN fit times of those CI runs differed three- to fivefold. The consistent explanation is that TensorFlow's oneDNN/Eigen kernels choose different vector instruction paths (AVX2, AVX-512 or ARM NEON) on different CPUs. These paths accumulate sums in different orders, and floating-point addition is not associative [10]. Thousands of Adam updates through ReLU networks then amplify last-bit differences into different trajectories. We had not logged the runners' CPU models at the time, so this attribution is strongly suggested rather than proven. The notebook now records CPU model and SIMD flags on every run.

The practical lessons are threefold:

- **Docker and `enable_op_determinism()` [11], [12] guarantee run-to-run identity on one machine**, which we confirmed, **but not across CPUs.**

- A single-run CNN difference below about 0.04 Macro-F1 should not be reported as an improvement.
- Our robust conclusions are the model ranking and per-class patterns, which held on every platform, not the third decimal place.

5. Conclusion

Main findings.

- **The CNN is the clear choice for this task.** It reached test Macro-F1 0.683 (accuracy 0.708), against 0.443 for the MLP and 0.386 for GaussianNB. It also used 5.7× fewer parameters than the MLP. Its advantage comes from inductive bias, not size: local 3×3 filters with shared weights match the texture-like, uncentred and locally correlated structure we found in the data. The cost is about 2.3× more training time per epoch than the MLP (≈ 87 s in total on 2 CPU cores). That is still modest, and prediction is fast.
- **The MLP is a poor use of parameters on images.** 605k weights bought only +0.03 Macro-F1 over a parameter-free Nearest Centroid, and the MLP completely failed on normal mucosa. It also showed an unstable configuration that collapsed to near-constant predictions.
- **GaussianNB is fast and transparent, but its independence assumption fails here.** It trains in under a second and its class models are easy to inspect. But summing 2,352 correlated log-likelihoods makes it both less accurate than Nearest Centroid and badly over-confident (97 % of posteriors above 0.999). Tuning `var_smoothing` cannot fix this.
- **The robust result is the ranking and the error pattern.** All models confuse the stromal and epithelial classes, with the CNN funnelling errors into TUM. Exact neural-network scores depend on the CPU: CNN Macro-F1 varied by 0.04 across platforms with identical pinned software.

Limitations.

- One seed and one validation split, with no confidence intervals. Differences under about 0.04 between CNN variants are within platform noise.
- A 12-epoch tuning budget that capped most CNN trials, which favours the faster learning rate.
- An epoch count transferred from a noisy validation curve.
- An image-level split without slide or patient IDs, so leakage between partitions cannot be ruled out.
- 28×28 inputs that remove gland architecture.
- Timings measured on a single 2-core machine.

Future work. Each item targets one of the limitations above.

1. **Repeat the final CNN and MLP with 5 seeds on two CPU types** and report mean ± s.d. This turns the cross-platform spread we observed into a proper uncertainty estimate and tells us which differences are real.
2. **Replace the fixed epoch count** with a cosine or step learning-rate decay, and choose the epoch from the seed-averaged validation curve. This should damp the epoch-to-epoch oscillation that made the arg-max fragile. Raise the epoch cap for the 3e-4 runs that were still improving.
3. **Give the CNN more context for NORM, STR and TUM** without a pre-trained model. Two options are a third convolution block, which grows the receptive field beyond 10×10, and rotation/flip augmentation, which suits rotation-invariant tissue. Both address the main remaining confusion. Success should be measured by NORM and STR recall and by TUM precision, not overall accuracy.
4. **Decorrelate the inputs for GaussianNB**, e.g. PCA to about 50–150 components (80–90 % of variance, Section 2.1). This directly tests whether the independence violation causes GNB's poor accuracy and calibration.
5. **Centre or standardise the MLP inputs and add batch normalisation** to test the dying-ReLU explanation of the collapsed trial.
6. **Use slide-level splits** if slide IDs are available, as in the original NCT-CRC-HE data [3], to obtain an unbiased estimate for unseen patients.

6. Reflection

530210074. Throughout A2, my understanding of machine learning evolved from focusing on individual model performance to appreciating the importance of experimental design and reproducibility. When developing the baseline and evaluation framework, I initially regarded simple models mainly as reference points. However, comparing their results with more complex approaches helped me understand why consistent metrics, controlled experimental conditions, and fair comparisons are essential for evaluating whether additional model complexity is truly justified.

My learning developed further during the Docker reproduction and final notebook integration. Observing differences in model results across computing environments challenged my assumption that fixing random seeds alone would guarantee identical outcomes. I learned to distinguish reproducibility within a controlled environment from exact numerical agreement across different systems, and to interpret small performance differences more cautiously. Ultimately, this project changed how I approach machine learning: rather than simply pursuing higher accuracy, I now place greater emphasis on understanding why models perform differently and ensuring that experimental conclusions are transparent, reliable, and reproducible.

540868894. Working on Gaussian Naive Bayes changed how I think about a model that is simple, fast and interpretable. The validation search over var_smoothing, which controls the variance estimates used in the class-conditional likelihoods, showed a long Macro-F1 plateau for small values and a decline once smoothing became large. This taught me that, for this model, hyperparameter tuning cannot compensate for an unsuitable modelling assumption. Neighbouring pixels and RGB channels are strongly correlated, so treating 2,352 pixel features as conditionally independent contributes to GaussianNB being over-confident (about 97% of its test predictions have a posterior above 0.999, yet only about 42% are correct) and to its scoring below the Nearest Centroid baseline.

I also learned that per-class results and the confusion matrix expose problems that overall scores hide. GaussianNB recognised a visually distinctive class such as adipose tissue well (recall 0.94) but rarely identified others such as smooth muscle (recall 0.08), which it most often confused with cancer-associated stroma. Keeping the validation search separate from the final test evaluation also helped me understand why the test set must not influence hyperparameter choices. Overall, GaussianNB taught me to ask which assumption a model makes about the data, and to check with per-class evidence whether that assumption actually holds.

550220550. Working on the MLP helped me understand that more parameters do not necessarily lead to better performance. Our model had approximately 605,000 parameters, yet achieved only a modest improvement over Nearest Centroid and performed substantially worse than the smaller CNN. This comparison taught me the importance of matching a model's structure to the data: flattening preserves pixel values, but a fully connected network lacks the local connectivity and weight sharing that help CNNs recognise tissue textures. The hyperparameter search also challenged the idea that dropout always improves generalisation. It reduced validation Macro-F1 in all four matched comparisons, while the similar training and validation losses suggested that insufficient learning was a greater concern than overfitting. Examining class-specific results further showed me why accuracy alone is inadequate: the MLP achieved 48.5% test accuracy but almost never recognised normal colon mucosa. In future work, I would test input centring and a longer training budget using validation data, and repeat experiments across seeds before treating small improvements as reliable.

550574604. Working on the CNN helped me understand how local receptive fields and shared filters capture spatial patterns in images. The per-class results showed that these advantages do not benefit every tissue class equally. Examining the confusion matrix taught me to consider Macro-F1 and class-specific recall alongside overall accuracy.

I also learned that hyperparameter effects depend on their interactions. Across the eight configurations, the benefit of dropout varied with the learning rate and filter width. Separating tuning from final training made the model easier to reproduce and integrate

into our group notebook. Using seed 5318 helped control randomness, while the limited search budget reminded me that our selected configuration was the best within the experiments we conducted.

7. AI Acknowledgement

Tools used. ChatGPT and Codex (OpenAI); Claude (Anthropic).

How they were used.

- *ChatGPT/Codex.* Interpreting the specification and template; debugging code for the individual models; writing the streaming Macro-F1 metric and timing callbacks; designing the Docker environment and the reproduction scripts; generating the Streamlit EDA dashboard; and helping test the Docker containers.
- *Claude.* Integrating the team's model code into the final notebook following the template; extending the exploratory analysis and search visualisations; testing the Docker containers against the Codex results; drafting the accompanying discussion; and helping members structure the main argument of the report.
- *Our own work.* We chose the models, search spaces and evaluation protocol. We checked all AI-produced code by executing it and checked every number in this report against the submitted notebook's outputs. The interpretations are our own. Each member's reflection was written by that member.

References

[1] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister and B. Ni, "MedMNIST v2 – A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, vol. 10, no. 41, 2023.

[2] J. Yang, R. Shi and B. Ni, "MedMNIST Classification Decathlon: A lightweight AutoML benchmark for medical image analysis," in *Proc. IEEE 18th Int. Symp. Biomedical Imaging (ISBI)*, 2021, pp. 191–195.

[3] J. N. Kather et al., "Predicting survival from colorectal cancer histology slides using deep learning: A retrospective multicenter study," *PLoS Medicine*, vol. 16, no. 1, e1002730, 2019.

[4] M. Macenko et al., "A method for normalizing histology slides for quantitative analysis," in *Proc. IEEE Int. Symp. Biomedical Imaging (ISBI)*, 2009, pp. 1107–1110.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[7] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[8] P. Domingos and M. Pazzani, "On the optimality of the simple Bayesian classifier under zero-one loss," *Machine Learning*, vol. 29, pp. 103–130, 1997.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[10] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[11] TensorFlow, "tf.config.experimental.enable_op_determinism," API documentation, v2.17. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/config/experimental/enable_op_determinism

[12] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, no. 239, 2014.